Set-A

1. **Write a function for Longest Valid Parentheses**
   Given a string containing just the characters '(' and ')', return the length of the longest valid (well-formed) parentheses
   Example 1: Input: s = "(()" Output: 2
   Explanation: The longest valid parentheses substring is "()".
   Example 2: Input: s = ")()())" Output: 4
   Explanation: The longest valid parentheses substring is "()()".

One of the Solution-

```
// Function to find the longest valid parentheses substring
int longestValidParentheses(char* s) {
    int n = strlen(s);
    int* stack = (int*)malloc((n + 1) * sizeof(int));
    int top = -1;

    // Initialize stack with base index
    stack[++top] = -1;
    int max_len = 0;

    for (int i = 0; i < n; i++) {
        if (s[i] == '(') {
            // Push index of '('
            stack[++top] = i;
        } else {
            // Pop the last '(' index
            top--;
            if (top < 0) {
                // Reset base index if stack is empty
                stack[++top] = i;
            } else {
                // Update max length
                int curr_len = i - stack[top];
                if (curr_len > max_len) {
                    max_len = curr_len;
                }
            }
        }
    }

    free(stack);
```

```
    return max_len;
}
```

****

2. **Swap Left and Right Children of Every Node (Mirror Tree)**
   Write a program swapTree() that takes a binary tree and swaps the left and right children of every node.

Sample Input:	**Sample Output:**
1 / \ 2 3 / \ 4 5	1 / \ 3 2 / \ 5 4

One of the Solution-

```
struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};

// Function to create a new node
struct Node* newNode(int data) {
    struct Node* node = (struct Node*)malloc(sizeof(struct Node));
    node->data = data;
    node->left = NULL;
    node->right = NULL;
    return node;
}

// Function to swap left and right children recursively
void swapTree(struct Node* root) {
    if (root == NULL) return;

    // Swap left and right pointers
    struct Node* temp = root->left;
    root->left = root->right;
    root->right = temp;

    // Recur for left and right subtrees
    swapTree(root->left);
```

```
    swapTree(root->right);
}

// Inorder traversal (to print tree)
void inorder(struct Node* root) {
    if (root == NULL) return;
    inorder(root->left);
    printf("%d ", root->data);
    inorder(root->right);
}
```

****

3. **Consider the following deque of characters where DEQUE is a circular array which is allocated six memory cells:**
   LEFT = 2,
   RIGHT = 4
   DEQUE: A, C, D
   Describe the deque while the following operations take place.
   (a) F is added to the right of the deque.
   (b). Two letters on the right are deleted.
   (c) K, L and M are added to the left of the deque.
   (d) One letter on the left is deleted.
   (e) R is added to the left of the deque.
   (f) S is added to the right of the deque.
   (g) T is added to the right of the deque.

One of the Solution-

Step	Operation	Deque Contents (Left → Right)	(LEFT, RIGHT)
0	Initial State	A, C, D	LEFT = 2, RIGHT = 4
(a)	Add F to right	A, C, D, F	LEFT = 2, RIGHT = 5
(b)	Delete 2 letters from right	A, C	LEFT = 2, RIGHT = 3
(c)	Add K, L, M to left	M, L, K, A, C	LEFT = 5, RIGHT = 3

(d)	Delete 1 letter from left	L, K, A, C	LEFT = 0, RIGHT = 3
(e)	Add R to left	R, L, K, A, C	LEFT = 5, RIGHT = 3
(f)	Add S to right	R, L, K, A, C, S	LEFT = 5, RIGHT = 4
(g)	Add T to right	**Not possible (Full)**	LEFT = 5, RIGHT = 5 (would overflow)

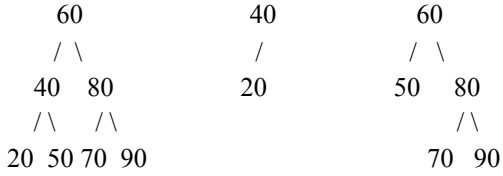
****

Set-B

1. **Given a Binary Search Tree (BST) and a key K, write a procedure to divide the BST into two trees:**
   a. One tree (T1) containing all keys < K
   b. Another tree (T2) containing all keys ≥ K

   The procedure must preserve the BST property in both resulting trees.

   Sample Input: K = 50 **Sample Output:**



One of the Solution-

```
// Recursive function to split BST into T1 (< K) and T2 (>= K)
void splitBST(struct Node* root, int K, struct Node** T1, struct Node** T2) {
  if (root == NULL) {
    *T1 = *T2 = NULL;
    return;
  }

  if (root->key < K) {
    // Root belongs to T1
    *T1 = root;
    splitBST(root->right, K, &((*T1)->right), T2);
```

```
    } else {
        // Root belongs to T2
        *T2 = root;
        splitBST(root->left, K, T1, &((*T2)->left));
    }
}
```

****

2. Consider the following deque where DEQUE is allocated 6 memory cells:

LEFT = 2, **RIGHT** = 5

DEQUE: [London. Berlin, Rome, Paris]

Describe the deque. including LEFT and RIGHT, as the following operations take place:

(a) Athens is added on the left.
(b) Two cities are deleted from the right.
(c) Madrid is added on the left.
(d) Moscow is added on the right.
(e) Two cities are deleted from the right.
(f) A city is deleted from the left.
(g) Oslo is added on the left.

One of the Solution-

Step	Operation	Deque Contents (Left → Right)	LEFT	RIGHT
0	Initial State	London, Berlin, Rome, Paris	2	5
(a)	Add Athens on left	Athens, London, Berlin, Rome, Paris	1	5
(b)	Delete 2 from right	Athens, London, Berlin	1	3
(c)	Add Madrid on left	Madrid, Athens, London, Berlin	0	3
(d)	Add Moscow on right	Madrid, Athens, London, Berlin, Moscow	0	4
(e)	Delete 2 from right	Madrid, Athens, London	0	2

(f)	Delete 1 from left	Athens, London	1	2
(g)	Add Oslo on left	Oslo, Athens, London	0	2

****

3. **Write a function to find Minimum Remove to Make Valid Parentheses**

Given a string s of '(' , ')' and lowercase English characters. Your task is to remove the minimum number of parentheses ('(' or ')', in any positions) so that the resulting parentheses string is valid and return any valid string.

Formally, a parentheses string is valid if and only if:

a. It is the empty string, contains only lowercase characters, or
b. It can be written as AB (A concatenated with B), where A and B are valid strings, or
c. It can be written as (A), where A is a valid string.

Example :
Input: s = "lee(t(c)o)de)"
Output: "lee(t(c)o)de"
Explanation: "lee(t(co)de)" , "lee(t(c)ode)" would also be accepted.

One of the Solution-

```
char* minRemoveToMakeValid(char* s) {
   int n = strlen(s);
   char* temp = (char*)malloc((n + 1) * sizeof(char));
   int balance = 0, j = 0;

   // First pass: remove extra ')'
   for (int i = 0; i < n; i++) {
      if (s[i] == '(') {
         balance++;
         temp[j++] = s[i];
      }
      else if (s[i] == ')') {
         if (balance > 0) {
            balance--;
            temp[j++] = s[i];
         }
         // else: skip unmatched ')'
      }
      else {
         temp[j++] = s[i];
      }
   }
   temp[j] = '\0';
```

```
    // Second pass: remove extra '('
    char* result = (char*)malloc((j + 1) * sizeof(char));
    int openToRemove = balance;
    int k = 0;

    for (int i = j - 1; i >= 0; i--) {
        if (temp[i] == '(' && openToRemove > 0) {
            openToRemove--; // skip this '('
        } else {
            result[k++] = temp[i];
        }
    }
    result[k] = '\0';

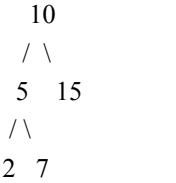
    // Reverse result to get correct order
    for (int i = 0; i < k / 2; i++) {
        char t = result[i];
        result[i] = result[k - 1 - i];
        result[k - 1 - i] = t;
    }

    free(temp);
    return result;
}
```

****

Set-B

1. **Count Leaf Nodes in a Binary Tree**
   Write a program that counts the number of leaf nodes in a binary tree. A leaf node is a node that has no left or right child.
   Sample Input:



   Sample Output:
   Number of leaf nodes = 3

One of the Solution-

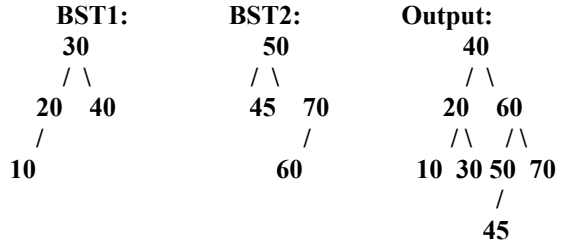
```
struct Node {
   int data;
   struct Node* left;
   struct Node* right;
};

// Function to create a new node
struct Node* newNode(int data) {
   struct Node* node = (struct Node*)malloc(sizeof(struct Node));
   node->data = data;
   node->left = node->right = NULL;
   return node;
}

// Function to count leaf nodes
int countLeafNodes(struct Node* root) {
   if (root == NULL)
      return 0;
   if (root->left == NULL && root->right == NULL)
      return 1;  // This is a leaf
   return countLeafNodes(root->left) + countLeafNodes(root->right);
}
```

****

2. **Given two Binary Search Trees (BSTs), write a procedure to merge them into a single balanced BST without inserting nodes one at a time from one tree into another. The procedure should take advantage of the fact that the inorder traversal of a BST gives a sorted sequence.**



One of the Solution-

```
// Inorder traversal to store elements in array
void storeInorder(struct Node* root, int inorder[], int* index) {
   if (root == NULL) return;
   storeInorder(root->left, inorder, index);
```

```
    inorder[(*index)++] = root->data;
    storeInorder(root->right, inorder, index);
}

// Merge two sorted arrays
int* mergeArrays(int a[], int n, int b[], int m, int* size) {
    int* merged = (int*)malloc((n + m) * sizeof(int));
    int i = 0, j = 0, k = 0;

    while (i < n && j < m) {
        if (a[i] < b[j]) merged[k++] = a[i++];
        else merged[k++] = b[j++];
    }
    while (i < n) merged[k++] = a[i++];
    while (j < m) merged[k++] = b[j++];

    *size = n + m;
    return merged;
}

// Build Balanced BST from sorted array
struct Node* sortedArrayToBST(int arr[], int start, int end) {
    if (start > end) return NULL;
    int mid = (start + end) / 2;
    struct Node* root = newNode(arr[mid]);
    root->left = sortedArrayToBST(arr, start, mid - 1);
    root->right = sortedArrayToBST(arr, mid + 1, end);
    return root;
}

// Merge two BSTs into one Balanced BST
struct Node* mergeBSTs(struct Node* root1, struct Node* root2) {
    int inorder1[100], inorder2[100];
    int n1 = 0, n2 = 0;

    // Get inorder traversal (sorted arrays)
    storeInorder(root1, inorder1, &n1);
    storeInorder(root2, inorder2, &n2);

    // Merge the two sorted arrays
    int size;
    int* merged = mergeArrays(inorder1, n1, inorder2, n2, &size);

    // Build Balanced BST from merged array
    struct Node* newRoot = sortedArrayToBST(merged, 0, size - 1);
```

```
    free(merged);
    return newRoot;
}
```

****

3. **A deque is implemented using a circular array of 6 memory cells.**
   Initially:
   LEFT = 0 RIGHT = 2
   DEQUE = [A, B, C]

Describe the deque. including LEFT and RIGHT, as the following operations take place:

(a) Add D to the right.
(b) Add E to the left.
(c) Delete two elements from the left.
(d) Add F and G to the right.
(e) Delete three elements from the right.
(f) Add H to the left.

One of the Solution-

Step	Operation	Deque Contents (Left → Right)	LEFT	RIGHT
0	**Initial State**	**A, B, C**	**0**	**2**
1	**Add D to the right**	**A, B, C, D**	**0**	**3**
2	**Add E to the left**	**E, A, B, C, D**	**5**	**3**
3	**Delete 2 from left**	**B, C, D**	**1**	**3**
4	**Add F, G to the right**	**B, C, D, F, G**	**1**	**5**
5	**Delete 3 from right**	**B, C**	**1**	**2**
6	**Add H to the left**	**H, B, C**	**0**	**2**

****